UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
THÀNH PHỐ HỒ CHÍ MINH
BAN CÔNG NGHỆ SỐ

ĐIỆN TOÁN ĐÁM MÂY

Chương 5 – Phát triển, triển khai và vận hành ứng dụng điện toán đám mây

Số tín chỉ: **3**

Giảng viên: TS. Lê Quốc Tuấn
Email : Tuan.le@ut.edu.vn

Dẫn nhập

Tình huống: triển khai nhanh một ứng dụng thật

- Doanh nghiệp cần ra mắt ứng dụng trong vài tuần thay vì vài tháng.
- Lượng người dùng biến động theo chiến dịch và mùa cao điểm.
- Ứng dụng phải có sao lưu, giám sát, bảo mật và khả năng rollback.
- Nhóm phát triển cần tự động hóa kiểm thử, đóng gói và triển khai.



Mục tiêu học tập

- Hiểu nguyên tắc phát triển ứng dụng cloud-native.
- Triển khai ứng dụng bằng container và Kubernetes ở mức nền tảng.
- Nắm quy trình DevOps, CI/CD, Infrastructure as Code và GitOps.
- Vận hành hệ thống bằng observability, SRE và tối ưu chi phí.
- Định hướng bài tập lớn: thiết kế một giải pháp cloud hoàn chỉnh.



Mục tiêu học tập

Bản đồ năng lực cần đạt sau chương 5

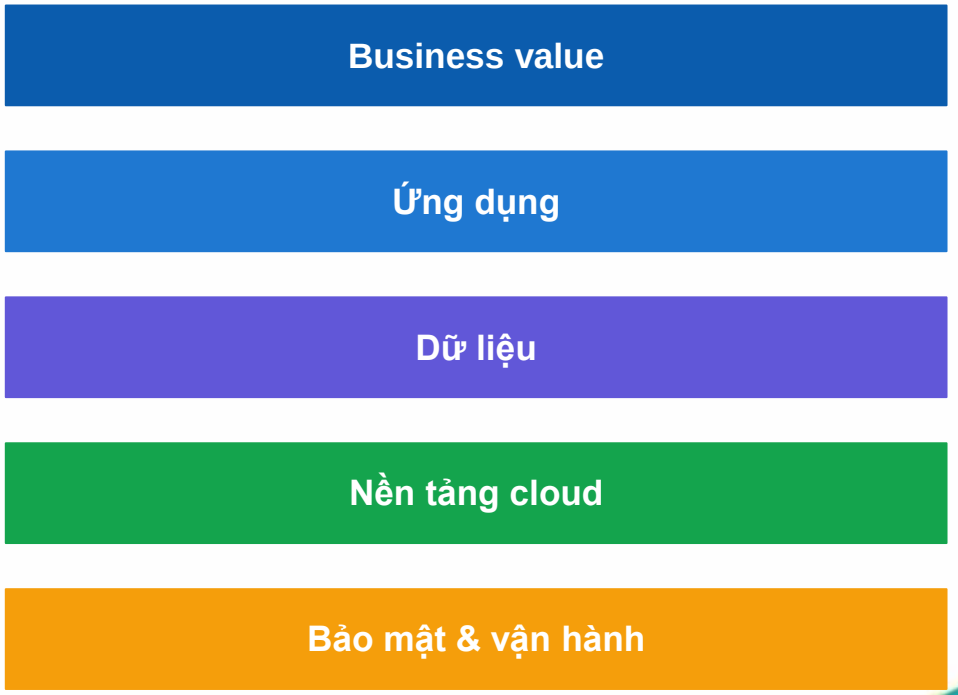


Thiết kế: chọn kiểu kiến trúc ứng dụng phù hợp với yêu cầu.

- Triển khai: đóng gói, đưa ứng dụng lên nền tảng cloud.
- Tự động hóa: CI/CD, IaC, GitOps giúp giảm thao tác thủ công.
- Vận hành: giám sát, xử lý sự cố, tối ưu hiệu năng và chi phí.
- Trình bày: giải thích được kiến trúc, quyết định kỹ thuật và rủi ro.

Từ "chạy được" đến "vận hành được"

- Ứng dụng chạy được mới chỉ là bước đầu.
- Ứng dụng cloud cần mở rộng, chịu lỗi, cập nhật an toàn và quan sát được.
- Mỗi thay đổi mã nguồn nên đi qua kiểm thử, đóng gói và kiểm soát phiên bản.
- Hạ tầng, bảo mật và chi phí phải được xem là một phần của thiết kế.



5.1. Nguyên tắc phát triển ứng dụng cloud-native

Thiết kế ứng dụng để tận dụng khả năng tự động hóa, co giãn và chịu lỗi của cloud.



Cloud-native là gì?

- Cloud-native là cách xây dựng và vận hành ứng dụng phù hợp môi trường cloud.
- Tập trung vào tự động hóa, co giãn, khả năng phục hồi và phát hành nhanh.
- Thường kết hợp container, microservices, API, CI/CD và observability.
- Không đồng nghĩa với việc mọi ứng dụng đều phải dùng microservices.

Ứng dụng truyền thống và ứng dụng cloud-native

Tiêu chí	Ứng dụng Truyền thống (Monolithic)	Ứng dụng Cloud-Native
Kiến trúc (Architecture)	**Monolithic (Khối đồng nhất)**. Các tính năng gắn kết chặt chẽ (Tightly-coupled).	**Microservices / Serverless**. Các dịch vụ nhỏ, độc lập, kết nối lỏng lẻo (Loosely-coupled).
Môi trường Triển khai	Máy chủ vật lý (Bare-metal) hoặc Máy ảo (VMs) cố định.	**Containers (Docker/Containerd)**, Kubernetes Pods, Serverless Functions.
Phụ thuộc Hạ tầng	Phụ thuộc sâu vào OS và cấu hình phần cứng cụ thể (Stateful).	**Hạ tầng Trừu tượng hóa (Infrastructure Agnostic)**. Chạy được trên bất kỳ Cloud nào (Stateless).
Khả năng Mở rộng (Scaling)	**Vertical Scaling (Scale Up)**: Tăng RAM/CPU của máy chủ. Tốn kém và có giới hạn.	**Horizontal Scaling (Scale Out)**: Tự động thêm/bớt các Container/Pod theo tải thực tế.
Quy trình Phát hành (Release)	Chu kỳ dài (hàng tháng/hàng quý). Mỗi lần deploy phải phát hành lại toàn bộ ứng dụng.	Chu kỳ cực ngắn (hàng ngày/hàng giờ). Triển khai độc lập từng Microservice qua **CI/CD**.
Độ tin cậy & Chịu lỗi	**Single Point of Failure**: Một lỗi nhỏ trong mã nguồn có thể làm **sập toàn bộ hệ thống**.	**Resilient / Fault-Tolerant**: Một Microservice sập không ảnh hưởng tới các thành phần khác. Auto-healing tự tạo lại Pod mới.
Quản trị Dữ liệu	Một CSDL tập trung duy nhất (Single Centralized RDBMS) cho toàn bộ ứng dụng.	**Polyglot Persistence**: Mỗi Microservice tự quản lý CSDL phù hợp nhất (SQL, NoSQL, Vector DB, Redis).
Phương thức Quản lý Hạ tầng	Cấu hình thủ công (Manual Configuration) hoặc dùng kịch bản Shell scripts.	**Infrastructure as Code (IaC)**: Quản lý hạ tầng bằng mã nguồn (Terraform, Helm, GitOps).

Kiến trúc nguyên khối

- Toàn bộ chức năng chính nằm trong một ứng dụng triển khai chung.
- Dễ phát triển ở giai đoạn đầu và dễ kiểm thử cục bộ.
- Khi lớn dần, khó mở rộng từng chức năng và khó phát hành độc lập.
- Phù hợp với hệ thống nhỏ hoặc nhóm phát triển còn ít kinh nghiệm.

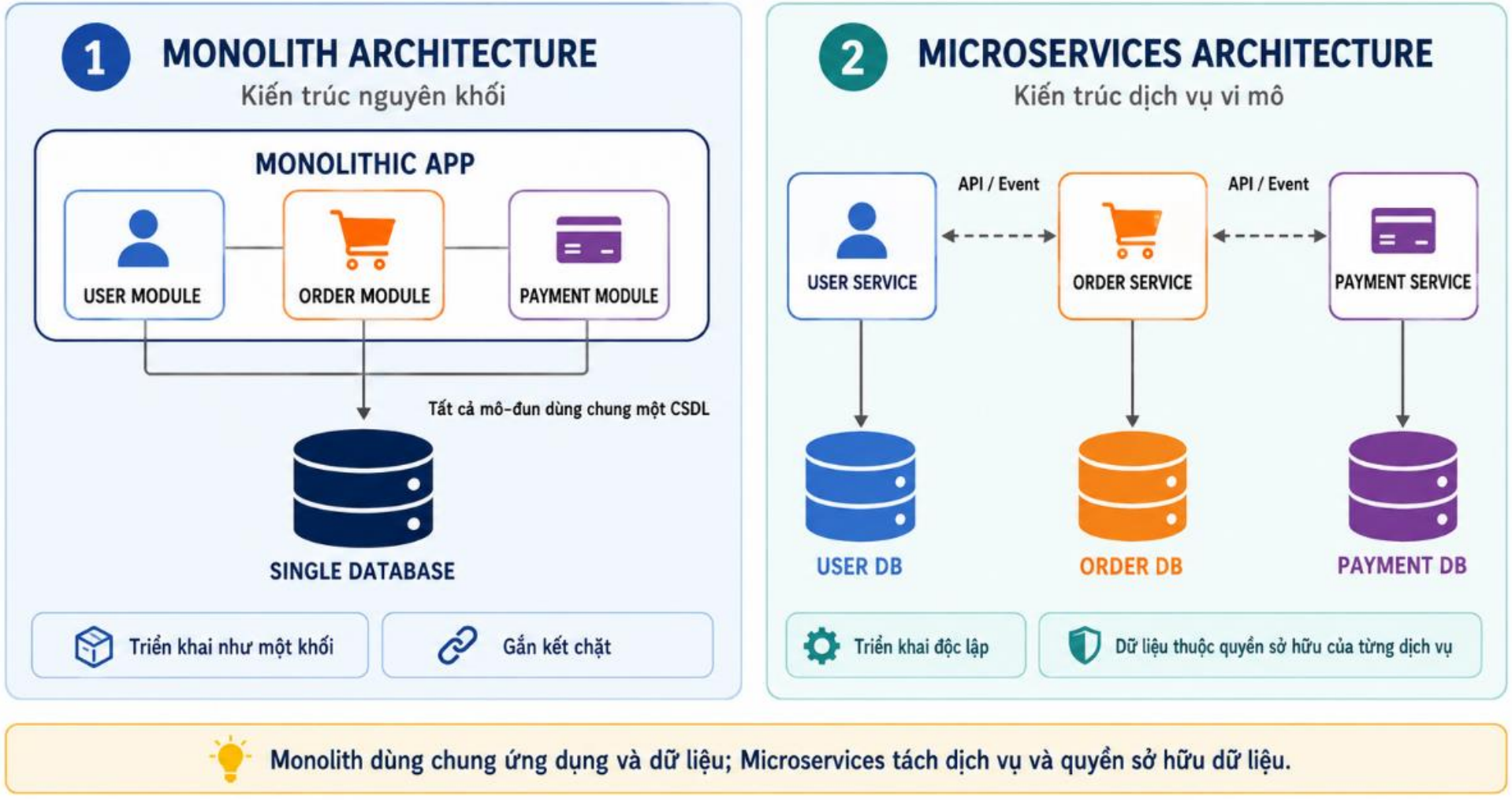


Kiến trúc microservices



- Ứng dụng được chia thành các dịch vụ nhỏ, độc lập theo năng lực nghiệp vụ.
- Mỗi dịch vụ có thể triển khai, mở rộng và quản lý vòng đời riêng.
- Giao tiếp qua API hoặc thông điệp.
- Đòi hỏi quản trị phức tạp hơn: quan sát, bảo mật, dữ liệu và giao tiếp.

So sánh Monolith và Microservices



UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

So sánh Monolith và Microservices

Tiêu chí	Monolith (Kiến trúc Khối)	Microservices (Kiến trúc Đa dịch vụ)
Độ phức tạp khởi đầu	**Thấp.** Rất dễ bắt đầu, viết code và định dạng dự án nhanh chóng.	**Rất cao.** Đòi hỏi hạ tầng phân tán, API Gateway, Service Discovery, CI/CD phức tạp ngay từ đầu.
Khả năng Mở rộng (Scaling)	**Scale hàng dọc (Vertical)** hoặc Scale toàn bộ khối (Scale Out toàn bộ App, lãng phí tài nguyên).	**Scale hàng ngang linh hoạt (Horizontal)**. Chỉ scale chính xác dịch vụ đang bị nghẽn (ví dụ: chỉ scale Payment Service ngày Flash Sale).
Môi trường & Công nghệ	**Đồng nhất (Single Tech Stack)**. Toàn bộ dự án phải dùng chung 1 ngôn ngữ/framework (ví dụ: Java Spring hoặc .NET).	**Đa dạng (Polyglot)**. Mỗi service có thể viết bằng ngôn ngữ tối ưu nhất (Python cho AI, Go cho High-throughput, Node.js cho I/O).
Độ tin cậy & Chịu lỗi	**Mất an toàn (Single Point of Failure)**. Một lỗi tràn bộ nhớ (Memory Leak) ở module tin tức có thể làm **sập toàn bộ hệ thống**.	**Tách biệt lỗi (Fault Isolation)**. Một service sập không làm ảnh hưởng trực tiếp đến các service khác (nếu áp dụng Circuit Breaker).
Triển khai (Deployment)	Chu kỳ chậm. Mỗi lần sửa 1 dòng code nhỏ đều phải test và deploy lại toàn bộ khối ứng dụng khổng lồ.	Triển khai độc lập, liên tục (CI/CD). Đội phát triển User Service có thể deploy 10 lần/ngày mà không ảnh hưởng đến Order Service.
Quản lý Dữ liệu	Dễ dàng duy trì tính toàn vẹn dữ liệu (**Transaction ACID**) trên một CSDL quan hệ tập trung.	Phức tạp (**Eventual Consistency**). Phải dùng các pattern như Saga, Outbox để quản lý giao dịch phân tán.
Đội ngũ & Tổ chức	Phù hợp với **Đội ngũ nhỏ (< 10-15 người)** làm việc trên một codebase.	Tối ưu cho **Tổ chức lớn với nhiều team độc lập** (Triết lý Two-Pizza Team của Amazon).
Kiểm thử & Debug	Đơn giản. Dễ dàng chạy End-to-End Test và Debug cục bộ trên máy dev.	Phức tạp. Cần các công cụ Distributed Tracing (OpenTelemetry, Jaeger) để truy vết request qua nhiều microservices.

Thành phần không trạng thái và có trạng thái



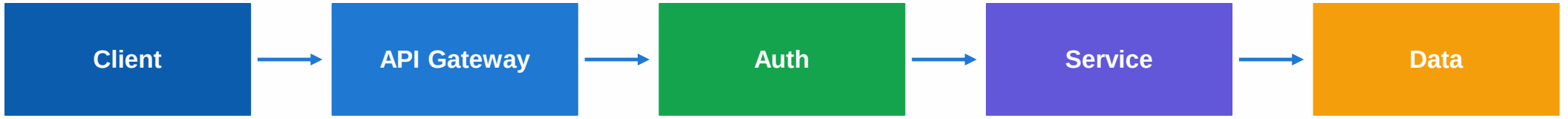
- Stateless: không lưu trạng thái phiên trong tiến trình ứng dụng.
- Stateful: cần lưu dữ liệu, phiên, hàng đợi hoặc trạng thái nghiệp vụ.
- Ứng dụng cloud ưu tiên stateless để dễ mở rộng ngang.
- Trạng thái nên đưa ra dịch vụ lưu trữ, cơ sở dữ liệu hoặc cache phù hợp.

Liên kết lỏng giữa các thành phần



- Giảm phụ thuộc trực tiếp giữa các dịch vụ.
- Dùng API, hàng đợi thông điệp, sự kiện hoặc pub/sub.
- Một thành phần lỗi không làm sập toàn bộ hệ thống.
- Dễ thay thế, mở rộng hoặc nâng cấp từng thành phần.

Thiết kế API-first



- API được xem là hợp đồng giữa các thành phần và nhóm phát triển.
- Xác định endpoint, dữ liệu vào/ra, mã lỗi và chính sách bảo mật sớm.
- Giúp frontend, backend, mobile và đối tác phát triển song song.
- Cần quản lý phiên bản API để tránh phá vỡ hệ thống đang chạy.

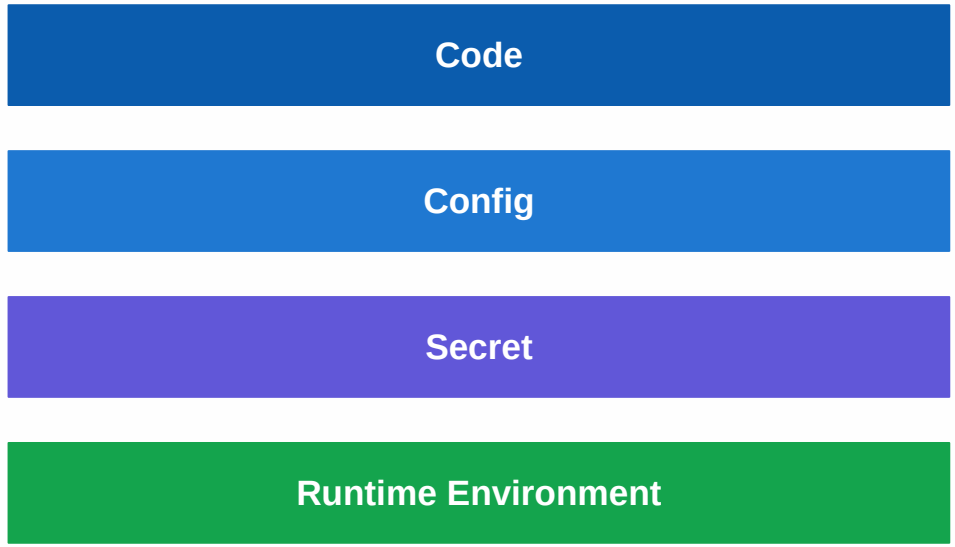
Kiến trúc hướng sự kiện



- Dịch vụ phản ứng với sự kiện thay vì gọi trực tiếp liên tục.
- Phù hợp xử lý bất đồng bộ, serverless, IoT và pipeline dữ liệu.
- Giảm phụ thuộc thời gian thực giữa producer và consumer.
- Cần quản lý thứ tự, trùng lặp, retry và lỗi xử lý.

Tách cấu hình khỏi mã nguồn

- Cùng một mã nguồn có thể chạy ở dev, test và production.
- Cấu hình được truyền qua biến môi trường, secret hoặc cấu hình tập trung.
- Mật khẩu, khóa API và token không được đặt trực tiếp trong mã nguồn.
- Giúp triển khai tự động và giảm lỗi cấu hình thủ công.

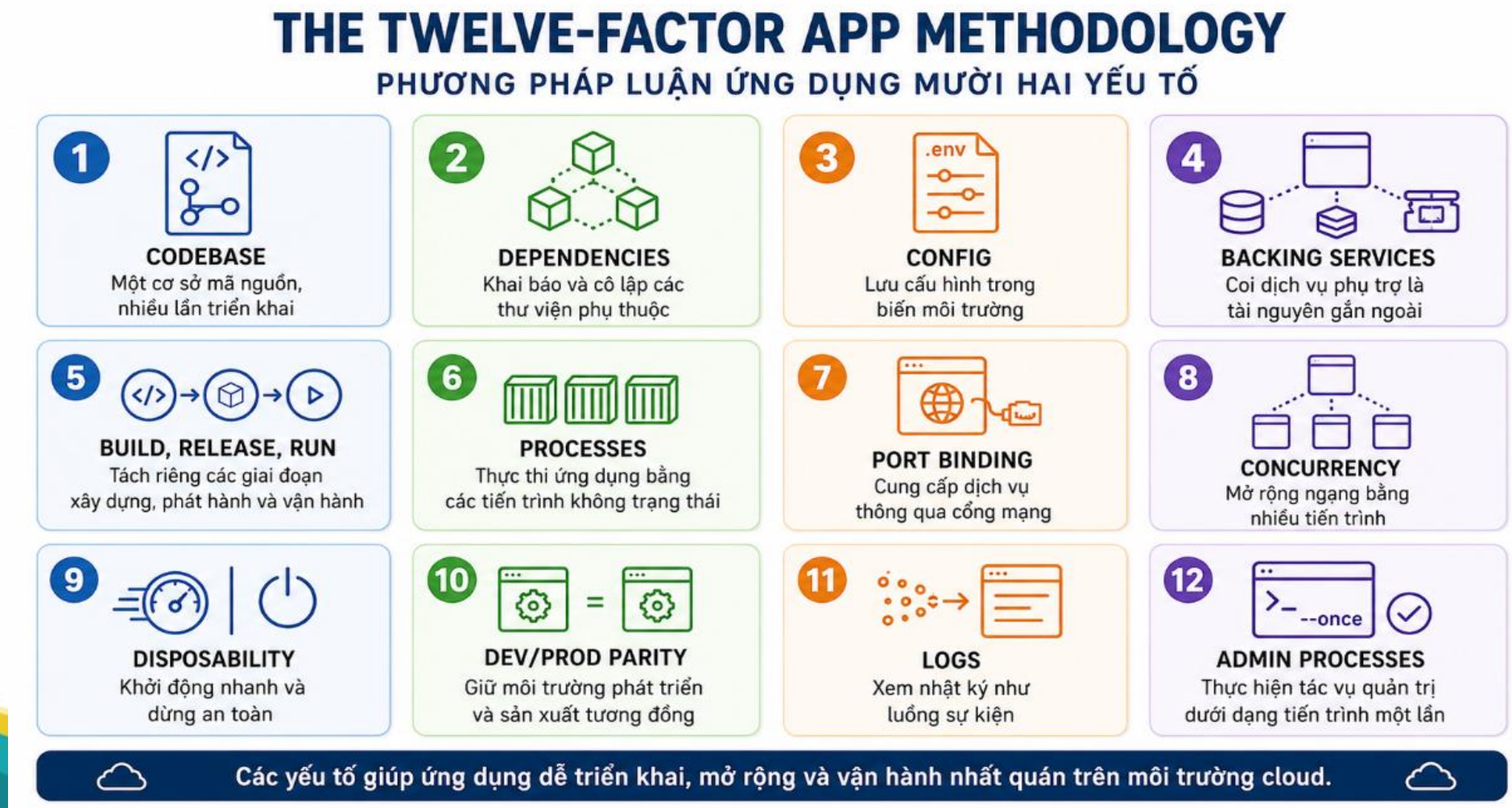


Hạ tầng bất biến



- Không sửa trực tiếp máy chủ đang chạy theo kiểu thủ công.
- Mỗi phiên bản được tạo mới từ image, template hoặc manifest.
- Khi cập nhật, thay thế tài nguyên thay vì chỉnh sửa tùy ý.
- Giúp rollback, kiểm soát phiên bản và tái lập môi trường dễ hơn.

Mười hai yếu tố ứng dụng



Mười hai yếu tố ứng dụng



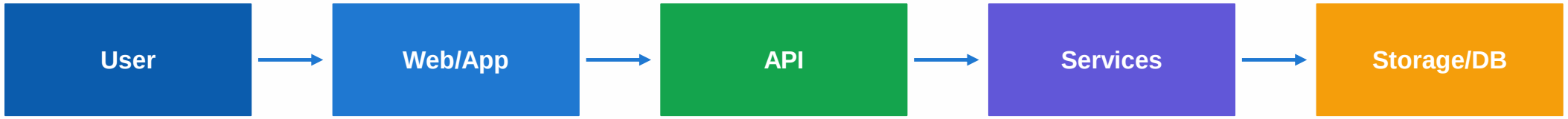
- Quản lý mã nguồn, phụ thuộc, cấu hình và dịch vụ nền rõ ràng.
- Xây dựng, phát hành và chạy được tách thành các giai đoạn.
- Ứng dụng không trạng thái, ghi log ra dòng sự kiện.
- Dễ triển khai lặp lại trên nhiều môi trường cloud.

Mẫu thiết kế khả năng phục hồi



- Retry có giới hạn khi lỗi tạm thời.
- Timeout để tránh treo tài nguyên vô hạn.
- Circuit breaker để cô lập dịch vụ lỗi.
- Bulkhead để giới hạn tác động lan truyền.
- Suy giảm chức năng có kiểm soát(Graceful degradation) để vẫn cung cấp chức năng tối thiểu.

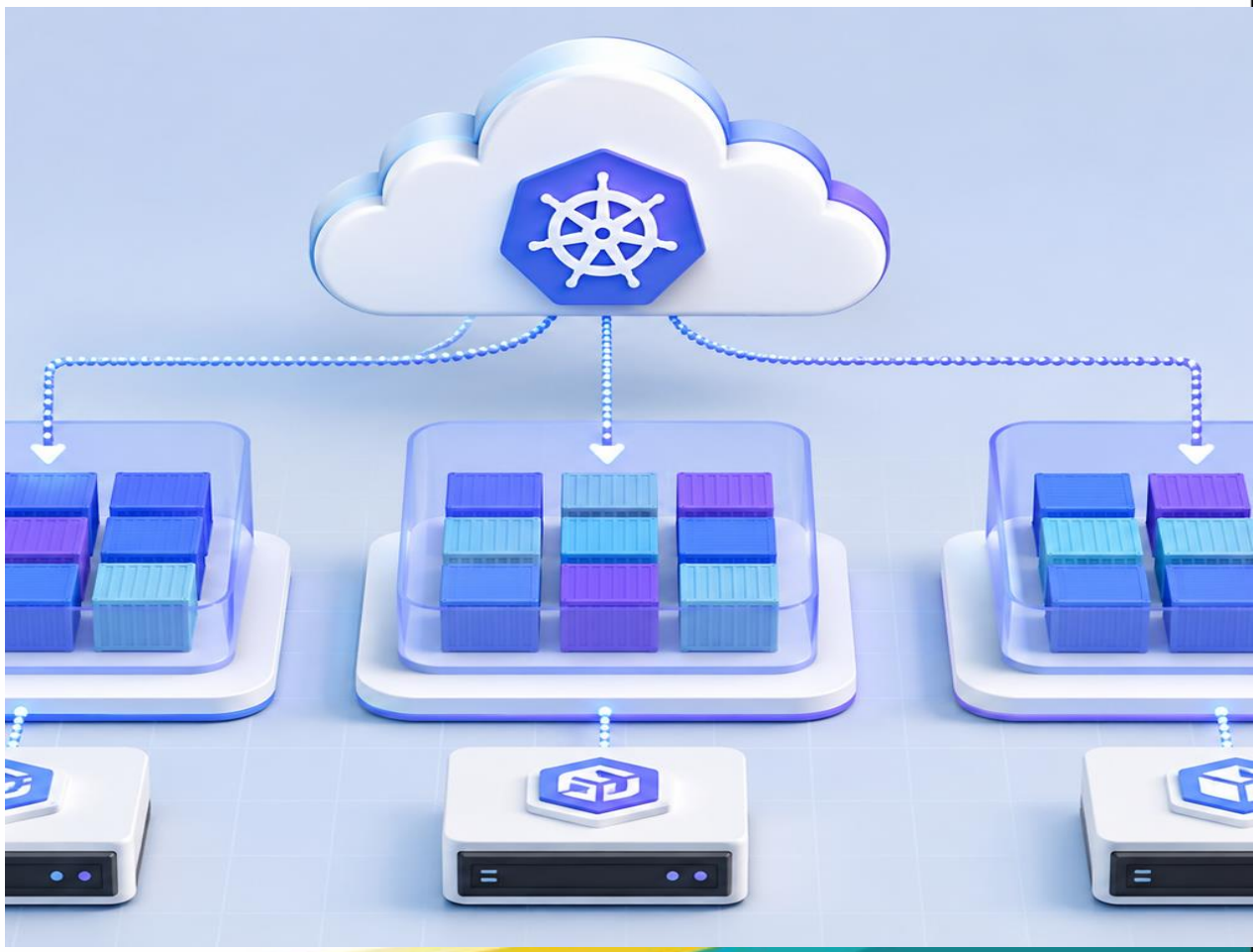
Tình huống: hệ thống LMS trên cloud



- Frontend phục vụ sinh viên và giảng viên qua Internet.
- Backend tách thành đăng nhập, khóa học, điểm, tài liệu và thông báo.
- File bài giảng lưu ở object storage; dữ liệu nghiệp vụ dùng managed database.
- Giám sát truy cập, lỗi, thời gian phản hồi và chi phí theo lớp dịch vụ.

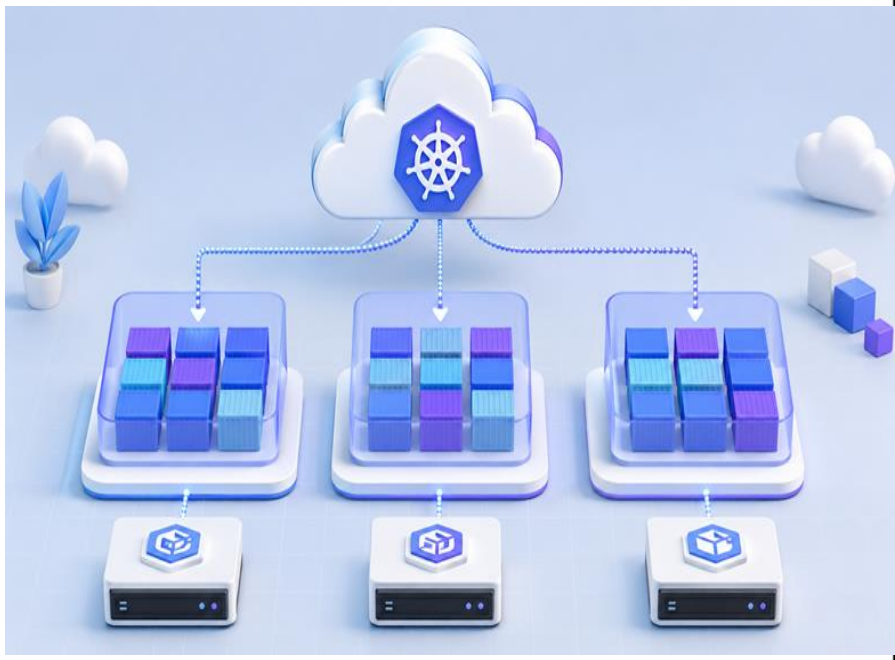
5.2. Đóng gói ứng dụng bằng container

Container giúp ứng dụng chạy nhất quán từ máy phát triển đến môi trường production.

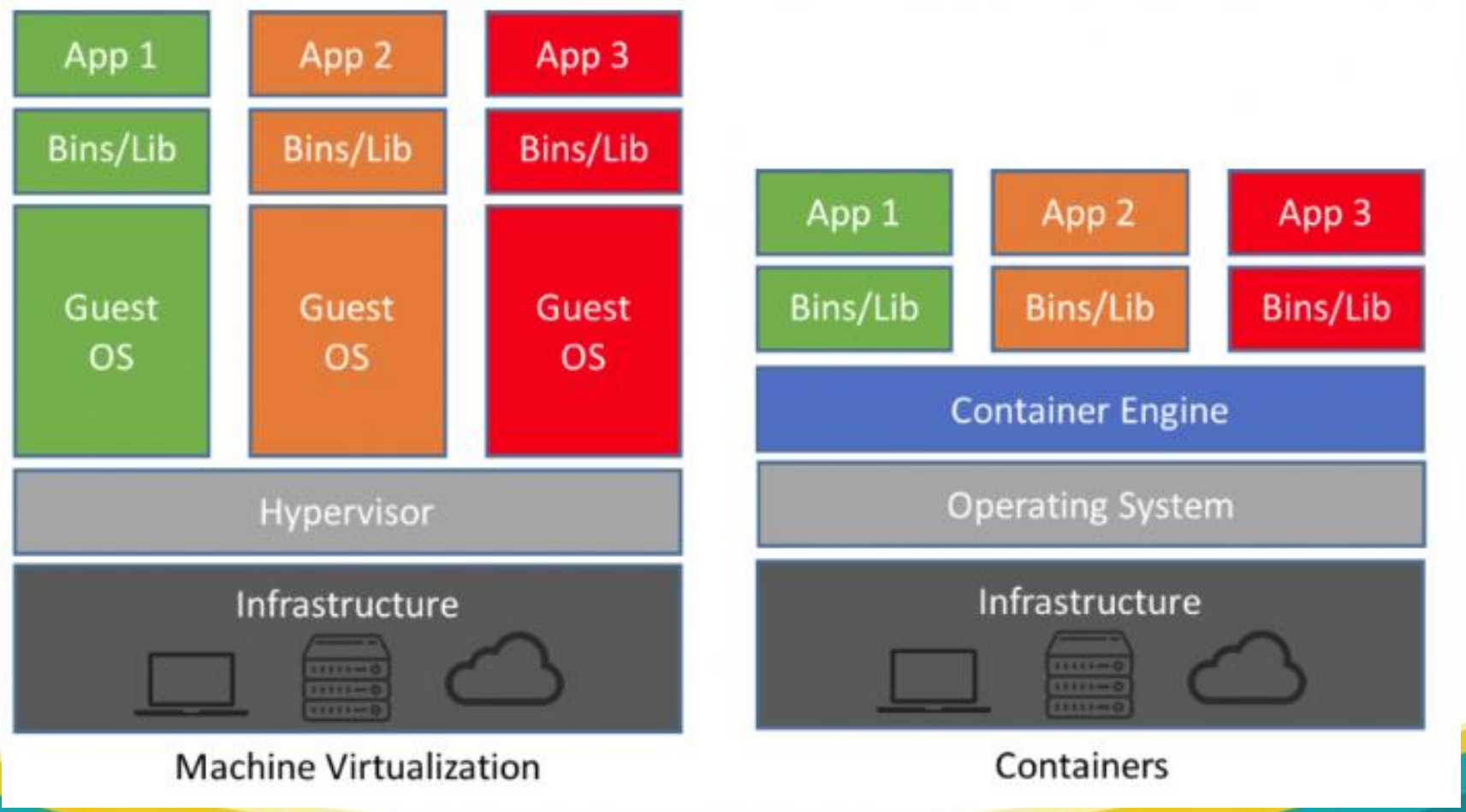


Vì sao cần container?

- Giảm lỗi “chạy được trên máy tôi” khi chuyển môi trường.
- Đóng gói ứng dụng cùng thư viện và cấu hình cần thiết.
- Khởi động nhanh hơn máy ảo trong nhiều trường hợp.
- Là nền tảng phổ biến để triển khai microservices và Kubernetes.



Container và máy ảo



Container và máy ảo

Tiêu chí	Máy ảo (Virtual Machine)	Container
Cơ chế Ảo hóa	Ảo hóa Phần cứng (Hardware Virtualization)	Ảo hóa Hệ điều hành (OS Virtualization)
Hệ điều hành (OS)	Mỗi VM chạy một **Guest OS riêng biệt** (Linux, Windows...).	Chia sẻ chung **Kernel của Host OS**.
Dung lượng (Size)	Rất lớn: từ vài **Gigabytes (GB)** đến hàng chục GB.	Rất nhẹ: từ vài **Megabytes (MB)** đến vài trăm MB.
Thời gian khởi động	Chậm: **Vài phút** (Phải khởi động toàn bộ tiến trình OS).	Cực nhanh: **Vài mili-giây đến vài giây**.
Hiệu năng (Performance)	Thấp hơn do tốn tài nguyên chạy Guest OS (*Overhead* cao).	Gần như **Native Speed** (Không bị tiêu tốn CPU/RAM cho Guest OS).
Mức độ Cô lập (Isolation)	**Cô lập cấp phần cứng (Hard Isolation)**. An toàn tuyệt đối.	**Cô lập cấp tiến trình (Process Isolation)**. Yếu hơn VM.
Mật độ Tài nguyên	Thấp: Một máy chủ chỉ chạy được vài chục VM.	Cực cao: Một máy chủ có thể chạy hàng trăm đến hàng ngàn Container.
Tính di động (Portability)	Hạn chế do phụ thuộc vào Hypervisor và cấu hình VM.	Tối ưu: Đóng gói chuẩn Image (OCI standard), *"Build once, run anywhere"*.

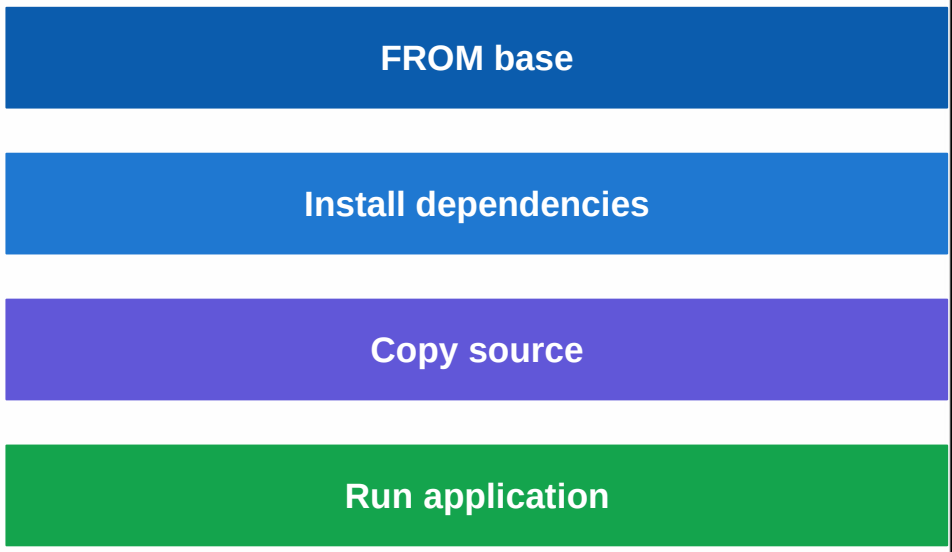
Ảnh container và registry



- Image là gói đóng băng ứng dụng và các phụ thuộc cần chạy.
- Registry là nơi lưu trữ, quản lý phiên bản và phân phối image.
- Tag image giúp phân biệt phiên bản, môi trường và bản phát hành.
- Cần kiểm soát quyền đẩy/kéo image và quét lỗ hổng.

Dockerfile và build image

- Dockerfile mô tả từng bước xây dựng image.
- Nên dùng base image đáng tin cậy và càng gọn càng tốt.
- Tách bước cài phụ thuộc và copy mã nguồn để tận dụng cache.
- Không đưa secret hoặc khóa truy cập vào image.



Lớp image và bộ nhớ đệm



- Mỗi lệnh build tạo một lớp image.
- Các lớp không đổi có thể tái sử dụng để tăng tốc build.
- Thay đổi file ở lớp sớm có thể làm mất cache các lớp sau.
- Tối ưu thứ tự lệnh giúp giảm thời gian build và dung lượng image.

Mạng container



- Container có thể giao tiếp qua bridge, overlay hoặc mạng của host.
- Cần hiểu port nội bộ và port ánh xạ ra ngoài.
- Không mở cổng không cần thiết ra Internet.
- Trong Kubernetes, Service và Ingress thay thế nhiều thao tác mạng thủ công.

Lưu trữ dữ liệu trong container



- Container có vòng đời ngắn; dữ liệu trong container có thể mất khi xóa container.
- Dữ liệu cần bền vững nên dùng volume, object storage hoặc database.
- Phân biệt log, cache, file upload và dữ liệu nghiệp vụ.
- Không nên lưu dữ liệu quan trọng trong filesystem tạm thời của container.

Docker Compose trong môi trường học tập

- Mô tả nhiều container trong một tệp cấu hình.
- Phù hợp mô phỏng ứng dụng gồm web, API, database, cache.
- Giúp sinh viên hiểu phụ thuộc giữa các dịch vụ trước khi học Kubernetes.
- Không thay thế hệ thống điều phối production ở quy mô lớn.



Quét lỗ hổng image container



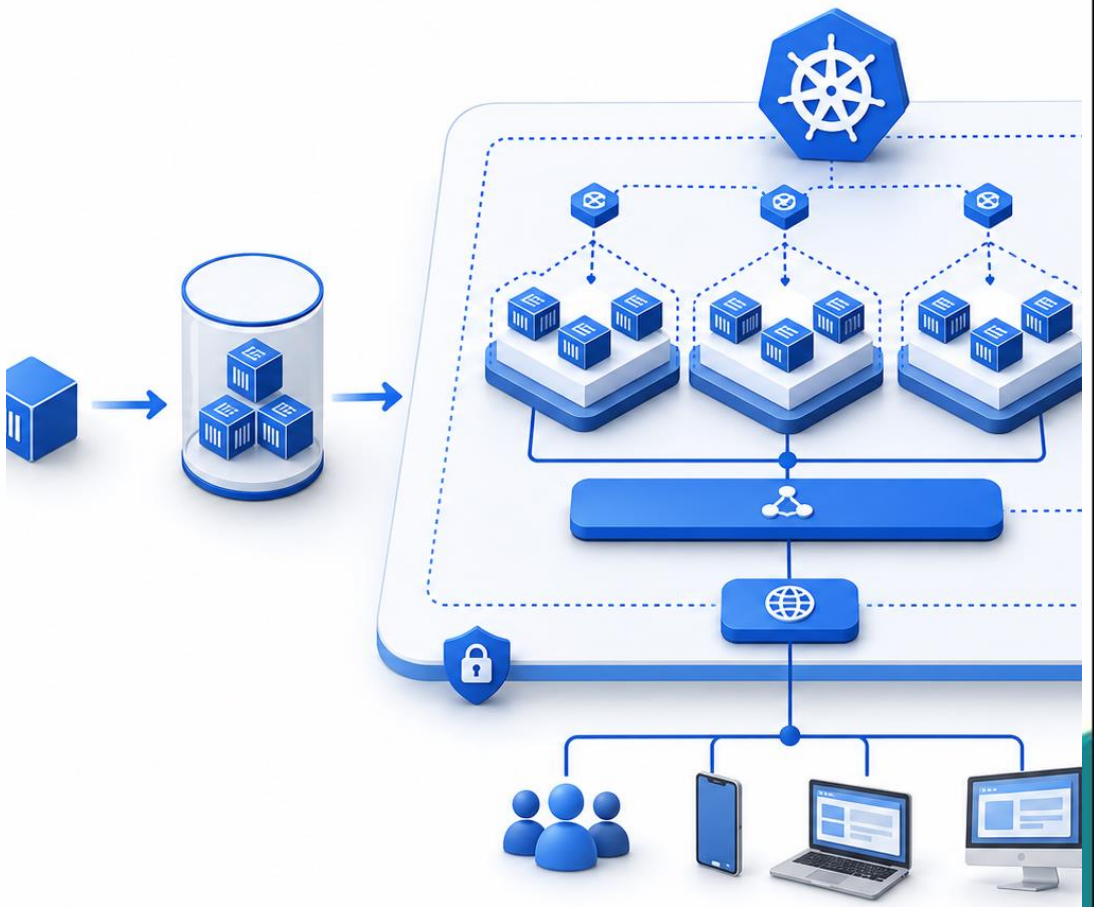
- Image có thể chứa thư viện lỗi thời hoặc lỗ hổng đã biết.
- Cần quét image trong pipeline trước khi đưa lên registry.
- Không chạy container bằng quyền root nếu không cần.
- Cần cập nhật base image định kỳ và theo dõi CVE.

Thực hành tốt với container

- Dùng image nhỏ, rõ nguồn gốc, cập nhật định kỳ.
- Đặt health check và log ra stdout/stderr.
- Một container nên ưu tiên một tiến trình chính.
- Pin phiên bản phụ thuộc để build có thể tái lập.
- Không lưu secret trong image hoặc repository.
- *Checklist nhanh: nhỏ – rõ nguồn – không secret – có health check – có version.*

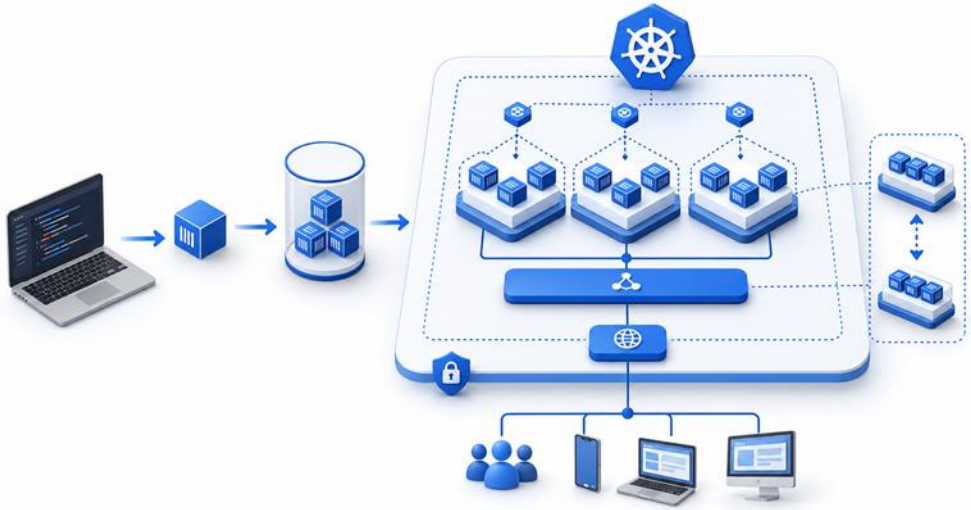
5.3. Điều phối ứng dụng với Kubernetes

- Kubernetes giúp triển khai, mở rộng và quản lý vòng đời ứng dụng container ở quy mô lớn.



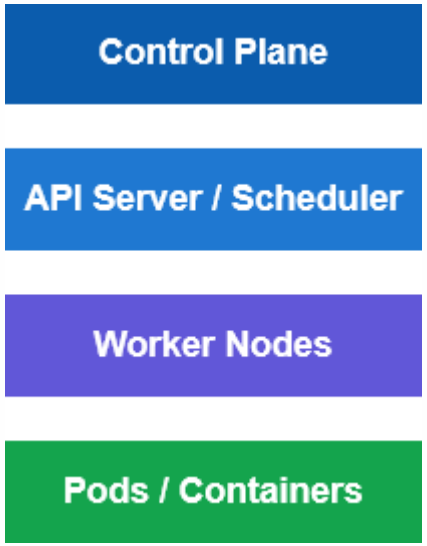
Vì sao cần điều phối container?

- Một ứng dụng production có thể gồm hàng chục hoặc hàng trăm container.
- Cần tự động đặt container lên node, phục hồi khi lỗi và mở rộng khi tải tăng.
- Cần quản lý cấu hình, secret, mạng, lưu trữ và cập nhật phiên bản.
- Kubernetes cung cấp mô hình khai báo để mô tả trạng thái mong muốn.



Kiến trúc tổng quan Kubernetes

- Control plane quản lý trạng thái cụm.
- Worker node chạy workload của ứng dụng.
- API Server là điểm giao tiếp chính.
- Scheduler chọn node phù hợp cho Pod.
- Controller liên tục điều chỉnh trạng thái thực tế về trạng thái mong muốn.



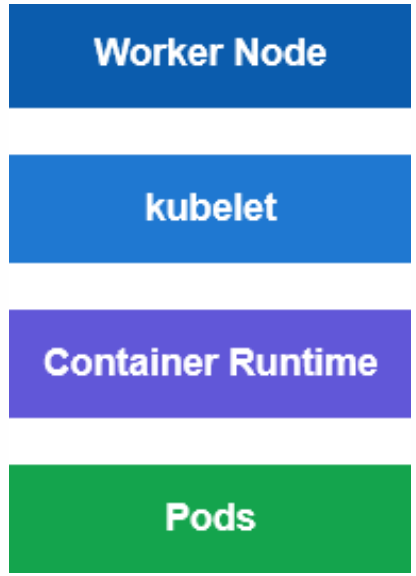
Control Plane



- API Server tiếp nhận yêu cầu từ kubectl, dashboard hoặc CI/CD.
- etcd lưu trạng thái cụm.
- Scheduler quyết định Pod chạy ở node nào.
- Controller Manager theo dõi và tự động sửa lệch trạng thái.

Worker Node và kubelet

- Worker node cung cấp CPU, RAM, mạng và lưu trữ để chạy Pod.
- kubelet nhận chỉ thị từ control plane và quản lý Pod trên node.
- Container runtime chạy container thực tế.
- Kube-proxy hoặc cơ chế mạng đảm nhiệm kết nối dịch vụ.

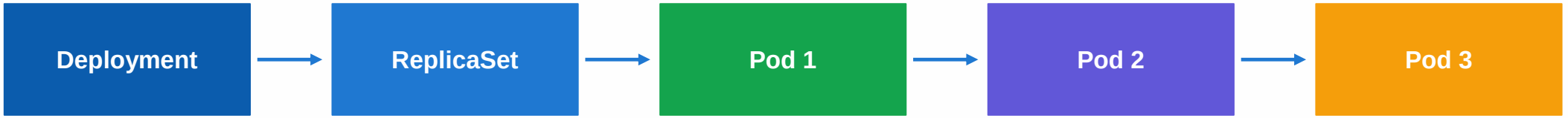


Pod



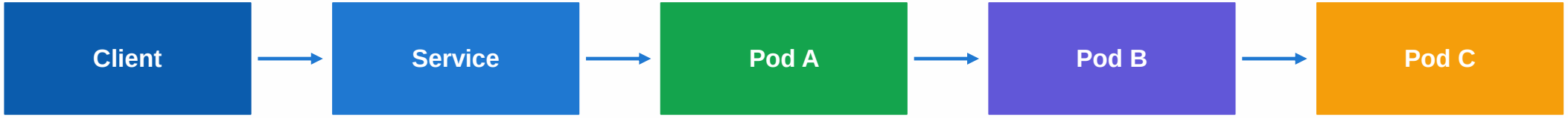
- Pod là đơn vị triển khai nhỏ nhất trong Kubernetes.
- Một Pod có thể chứa một hoặc nhiều container chia sẻ network namespace.
- Pod thường có vòng đời ngắn và có thể bị thay thế.
- Không nên truy cập Pod trực tiếp bằng IP cố định trong thiết kế ứng dụng.

Deployment và ReplicaSet



- Deployment mô tả phiên bản ứng dụng và chiến lược cập nhật.
- ReplicaSet duy trì số lượng Pod mong muốn.
- Khi Pod lỗi, controller tạo Pod mới để bù vào.
- Deployment hỗ trợ rolling update và rollback.

Service



- Service cung cấp địa chỉ ổn định để truy cập nhóm Pod.
- Các Pod phía sau Service có thể thay đổi nhưng tên Service giữ nguyên.
- ClusterIP dùng trong cụm, NodePort/LoadBalancer dùng để xuất dịch vụ ra ngoài.
- Service giúp cân bằng tải nội bộ đến các Pod.

Ingress



- Ingress quản lý truy cập HTTP/HTTPS từ bên ngoài vào cụm.
- Có thể định tuyến theo tên miền hoặc đường dẫn.
- Thường kết hợp TLS, chứng chỉ và Ingress Controller.
- Phù hợp đưa nhiều dịch vụ web ra ngoài qua một điểm vào.

Namespace

- Namespace chia cụm thành các không gian logic.
- Hỗ trợ tách môi trường, nhóm dự án hoặc nhóm người dùng.
- Có thể áp dụng quota, policy và RBAC theo namespace.
- Không phải là ranh giới bảo mật tuyệt đối nếu cấu hình yếu.



ConfigMap và Secret



- ConfigMap lưu cấu hình không nhạy cảm.
- Secret lưu dữ liệu nhạy cảm như mật khẩu, token, khóa API.
- Ứng dụng có thể nhận cấu hình qua biến môi trường hoặc volume.
- Secret cần được mã hóa và giới hạn quyền truy cập.

Persistent Volume



- Persistent Volume cung cấp lưu trữ bền vững cho workload stateful.
- Persistent Volume Claim là yêu cầu sử dụng lưu trữ từ ứng dụng.
- StorageClass cho phép cấp phát động theo loại lưu trữ.
- Cần hiểu hiệu năng, backup và khả năng di chuyển dữ liệu.

Health Check: Liveness và Readiness



- Liveness probe phát hiện container cần khởi động lại.
- Readiness probe xác định Pod đã sẵn sàng nhận traffic chưa.
- Startup probe hỗ trợ ứng dụng khởi động lâu.
- Probe sai có thể gây vòng lặp restart hoặc đưa traffic vào ứng dụng chưa sẵn sàng.

Mở rộng ứng dụng trong Kubernetes



- Manual scaling thay đổi số replica bằng lệnh hoặc manifest.
- Horizontal Pod Autoscaler tự mở rộng theo CPU, memory hoặc metric tùy chỉnh.
- Cluster Autoscaler tăng/giảm node khi thiếu hoặc dư tài nguyên.
- Cần đặt request/limit phù hợp để autoscaling hoạt động chính xác.

Rolling Update và Rollback



- Rolling update thay thế Pod cũ bằng Pod mới từng phần.
- Giảm thời gian gián đoạn khi phát hành phiên bản mới.
- Rollback quay lại phiên bản ổn định khi phát hiện lỗi.
- Cần kết hợp readiness probe và quan sát metric để phát hành an toàn.

Quy trình triển khai lên Kubernetes



- Viết mã nguồn và Dockerfile.
- Build image và đẩy lên registry.
- Viết manifest Deployment, Service, Ingress.
- Áp dụng manifest vào cụm.
- Theo dõi Pod, log, metric và trạng thái rollout.

Lỗi thường gặp khi học Kubernetes

- ImagePullBackOff do sai tên image, tag hoặc quyền registry.
- CrashLoopBackOff do ứng dụng lỗi hoặc thiếu cấu hình.
- Service không truy cập được do sai selector hoặc port.
- Pod pending do thiếu tài nguyên hoặc node không phù hợp.
- Secret/ConfigMap chưa mount đúng vào ứng dụng.

5.4. DevOps và quy trình CI/CD

- Tự động hóa vòng đời phần mềm để phát hành nhanh, ổn định và có kiểm soát.



DevOps là gì?

- DevOps kết hợp phát triển, vận hành, tự động hóa và văn hóa cộng tác.
- Mục tiêu là rút ngắn chu kỳ phát hành nhưng vẫn bảo đảm chất lượng.
- Không phải chỉ là một công cụ hoặc một vị trí công việc.
- DevOps tốt cần đo lường, phản hồi nhanh và cải tiến liên tục.

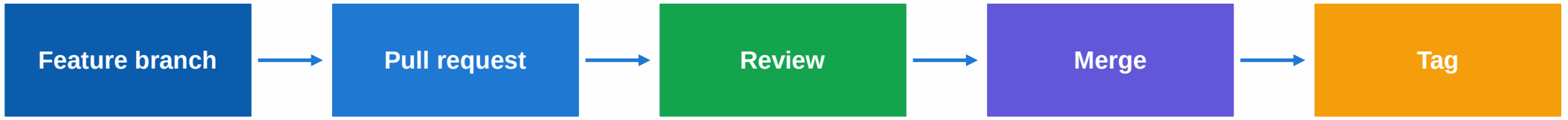


Tổng quan CI/CD



- CI: tích hợp liên tục, kiểm tra mã nguồn thường xuyên.
- CD: phân phối hoặc triển khai liên tục sau khi vượt qua kiểm thử.
- Pipeline gồm build, test, scan, package, deploy và verify.
- Mỗi bước nên có điều kiện dừng khi phát hiện lỗi.

Quản lý mã nguồn bằng Git



- Mỗi thay đổi cần được ghi nhận, xem xét và có lịch sử rõ ràng.
- Branching strategy giúp nhóm phát triển song song.
- Pull request hỗ trợ review mã nguồn và thảo luận kỹ thuật.
- Tag và release giúp truy vết phiên bản triển khai.

Giai đoạn Build



- Biên dịch hoặc đóng gói ứng dụng.
- Tạo container image hoặc artifact.
- Ghi nhận metadata: commit hash, version, thời gian build.
- Build phải tái lập được trên môi trường pipeline.

Kiểm thử tự động



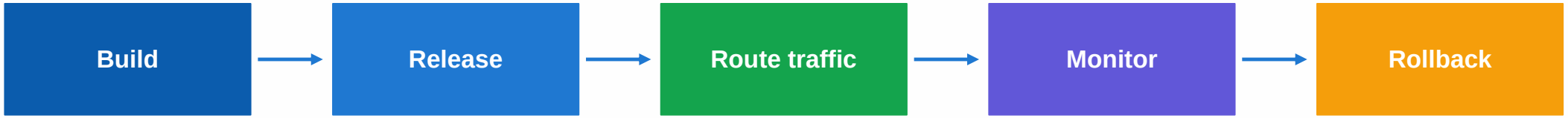
- Unit test kiểm tra từng đơn vị mã nguồn.
- Integration test kiểm tra tương tác giữa thành phần.
- End-to-end test mô phỏng luồng người dùng.
- Security test và quality gate giúp ngăn lỗi nghiêm trọng đi tiếp.

Artifact Registry



- Lưu trữ container image, package, binary hoặc chart triển khai.
- Gắn version rõ ràng và không ghi đè tag production tùy tiện.
- Phân quyền truy cập và lưu log tải/xóa artifact.
- Registry là nguồn phân phối đáng tin cậy cho môi trường triển khai.

Chiến lược triển khai ứng dụng



- Rolling update: thay thế dần phiên bản cũ.
- Blue–Green: chạy song song hai môi trường và chuyển traffic.
- Canary: đưa phiên bản mới cho một phần nhỏ người dùng.
- A/B Testing: so sánh trải nghiệm hoặc tính năng theo nhóm người dùng.

Rolling Update



- Cập nhật từng phần, duy trì một số instance cũ trong lúc triển khai.
- Ít yêu cầu hạ tầng bổ sung hơn blue–green.
- Cần readiness check tốt để tránh đưa traffic vào instance lỗi.
- Khó hơn khi thay đổi không tương thích về schema dữ liệu.

Blue–Green Deployment



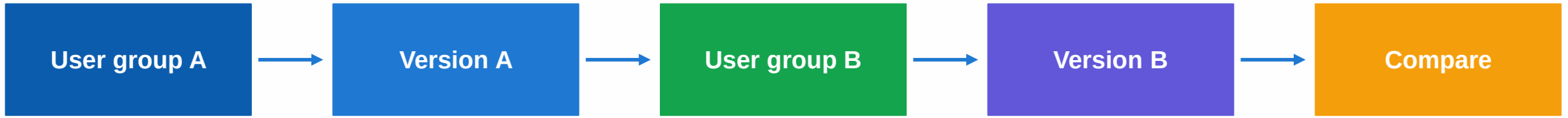
- Blue là môi trường đang chạy, Green là môi trường phiên bản mới.
- Kiểm thử Green trước khi chuyển traffic.
- Rollback nhanh bằng cách chuyển traffic lại Blue.
- Tốn thêm tài nguyên vì cần hai môi trường song song.

Canary Deployment



- Phát hành phiên bản mới cho một tỷ lệ nhỏ người dùng hoặc request.
- Theo dõi lỗi, latency, CPU, memory và chỉ số nghiệp vụ.
- Tăng dần tỷ lệ nếu hệ thống ổn định.
- Tự động rollback nếu metric vượt ngưỡng rủi ro.

A/B Testing



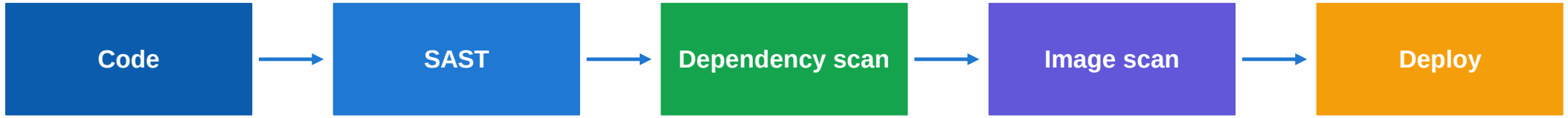
- Dùng để so sánh hai biến thể tính năng hoặc giao diện.
- Mục tiêu thường là tối ưu trải nghiệm hoặc chỉ số kinh doanh.
- Cần phân nhóm người dùng và thu thập dữ liệu chính xác.
- Không nên nhầm với canary deployment, vốn tập trung giảm rủi ro kỹ thuật.

Rollback và kiểm soát phiên bản



- Mọi bản triển khai phải có khả năng quay lui hoặc khắc phục nhanh.
- Cần biết phiên bản mã, image, database schema và cấu hình đang chạy.
- Không thay đổi thủ công production ngoài pipeline nếu không có ghi nhận.
- Rollback ứng dụng đơn giản hơn rollback dữ liệu.

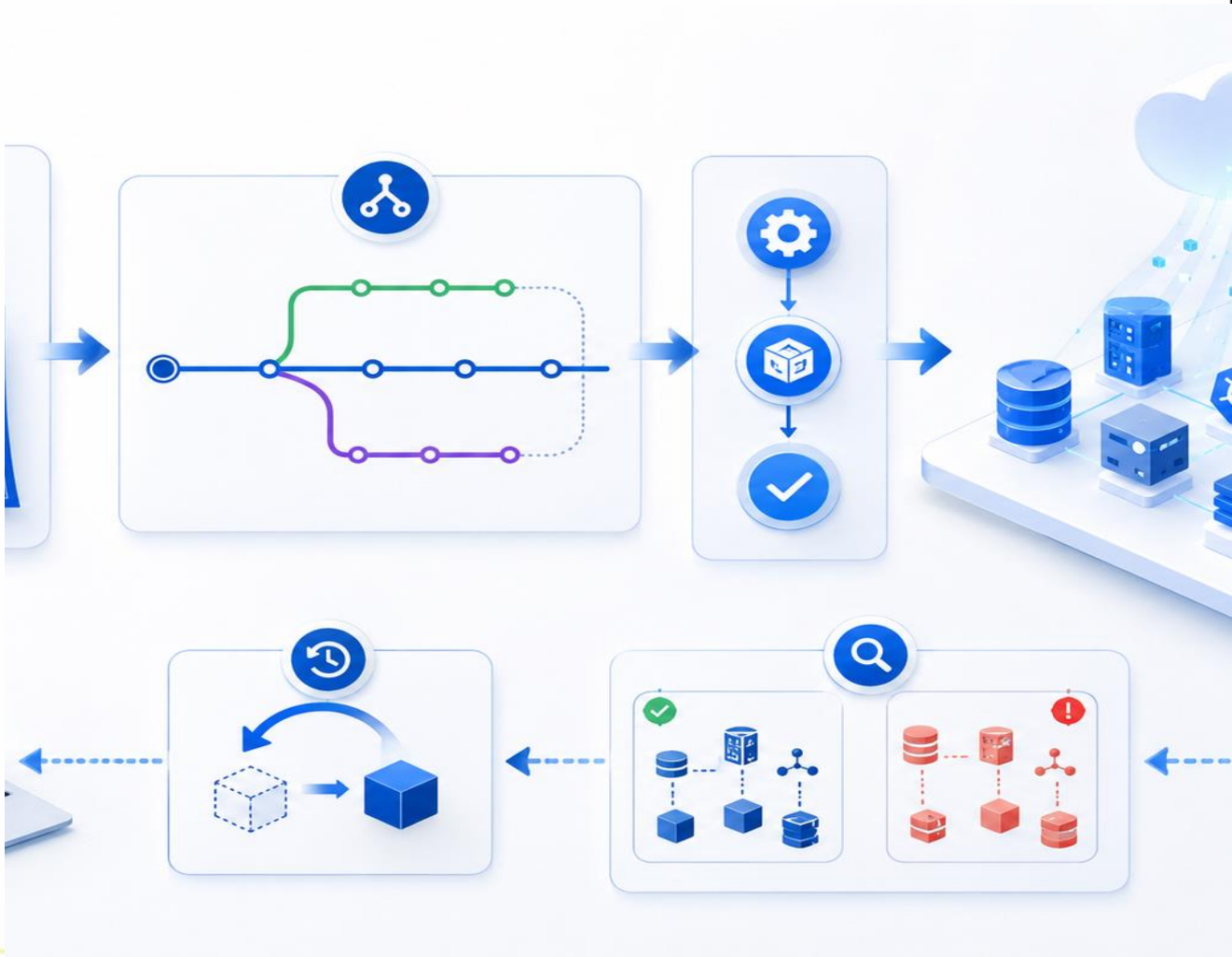
DevSecOps



- Đưa bảo mật vào sớm trong pipeline thay vì kiểm tra cuối cùng.
- Quét dependency, image, secret và cấu hình hạ tầng.
- Áp dụng policy tự động và phê duyệt thay đổi rủi ro cao.
- Ghi nhận bằng chứng kiểm tra để phục vụ kiểm toán.

5.5. Infrastructure as Code và GitOps

- Mô tả hạ tầng bằng mã để triển khai lặp lại, kiểm soát phiên bản và giảm lỗi thủ công.



Infrastructure as Code là gì?

- Hạ tầng được mô tả bằng tệp cấu hình hoặc mã khai báo.
- Có thể tạo, sửa, xóa tài nguyên cloud bằng pipeline tự động.
- Tệp hạ tầng được lưu version trong Git như mã nguồn ứng dụng.
- Giúp tái tạo môi trường dev, test và production nhất quán.



Khai báo và mệnh lệnh

- Trong cấu trúc code điện toán đám mây và Hạ tầng dưới dạng Mã (Infrastructure as Code - IaC), **Khai báo (Declarative)** và **Mệnh lệnh (Imperative)** là hai mô hình tư duy lập trình cốt lõi quyết định cách thức hệ thống tạo lập, thay đổi và duy trì tài nguyên hạ tầng.

Bảng So sánh Cốt lõi: Khai báo vs. Mệnh lệnh

Tiêu chí	Mô hình Khai báo (Declarative)	Mô hình Mệnh lệnh (Imperative)
Triết lý cốt lõi	Định nghĩa **"CÁI GÌ"** (Trạng thái đích mong muốn - *What*).	Chỉ định **"NHƯ THẾ NÀO"** (Các bước thực thi - *How*).
Cơ chế vận hành	Mô tả End-state; Engine tự tính toán khoảng cách (Diff) để tự điều chỉnh.	Thực thi danh sách câu lệnh theo trình tự tuyến tính từ trên xuống dưới.
Tính đồng đẳng (Idempotency)	**Tự nhiên (Native)**. Chạy lại script nhiều lần vẫn cho ra đúng 1 kết quả như cũ.	**Thủ công**. Bắt buộc viết code tự kiểm tra trạng thái trước khi thực thi để tránh tạo trùng.
Quản lý State	Phụ thuộc vào File lưu trạng thái (Terraform State / K8s etcd) để so sánh.	Không quản lý State tập trung; dựa vào log thực thi của từng câu lệnh.
Khả năng bảo trì	Dễ đọc, dễ kiểm toán (Audit), tối ưu cho chu trình CI/CD và GitOps.	Phức tạp khi scale lớn; dễ đứt gãy nếu một bước trung gian bị lỗi.
Công cụ tiêu biểu	Terraform (HCL), AWS CloudFormation, Kubernetes Manifests (YAML).	AWS CLI, Azure CLI, Bash/PowerShell scripts, Python (Boto3).

Công cụ IaC phổ biến

Công cụ	Nhà phát triển	Ngôn ngữ sử dụng	Mô hình	Trường hợp sử dụng tối ưu
Terraform	HashiCorp	HCL (HashiCorp Configuration Language)	Declarative	Tiêu chuẩn ngành cho Multi-Cloud provisioning
OpenTofu	Linux Foundation	HCL (Open-source fork)	Declarative	Thay thế mã nguồn mở hoàn toàn cho Terraform
Pulumi	Pulumi	TypeScript, Python, Go, C#, Java	Hybrid (Code-based)	Quản lý Cloud bằng ngôn ngữ lập trình thực sự
AWS CDK	Amazon	TypeScript, Python, Java, C#	Code ➔ CloudFormation	Triển khai hạ tầng AWS chuyên sâu bằng code
Bicep	Microsoft	Bicep DSL	Declarative	Thay thế ARM JSON gọn nhẹ trên Azure
Ansible	Red Hat / IBM	YAML	Declarative / Imperative	Quản lý cấu hình, cài phần mềm (Agentless)
Crossplane	CNCF	YAML (Kubernetes CRD)	Declarative	Biến Kubernetes thành Control Plane quản lý Cloud

Tệp trạng thái hạ tầng (Infrastructure State File)

Tiêu chí	Giải pháp Tiêu chuẩn	Mục đích / Tác dụng
Lưu trữ Tập trung (Remote Backend)	Dùng AWS S3, Azure Blob, GCP Bucket hoặc Terraform Cloud. **Cấm commit State vào Git.**	Đảm bảo tính nhất quán cho toàn bộ đội ngũ kỹ sư.
Khóa Trạng thái (State Locking)	Kết hợp DynamoDB (AWS), Native Blob Lease (Azure), hoặc Redis.	Ngăn chặn nhiều người cùng can thiệp hạ tầng một lúc.
Mã hóa Tuyệt đối (At-Rest & In-Transit)	Bật mã hóa KMS (AES-256) cho Bucket lưu trữ và bắt buộc TLS 1.3 khi truyền tải.	Tránh lộ lọt Passwords/Secrets lưu vết bên trong State.
Phân chia Phạm vi (State Isolation)	Tách nhỏ State File theo Môi trường (Dev/Prod) và theo Tầng (Network/App/Data).	Thu hẹp rủi ro (Blast Radius); sự cố ở một module không làm sập toàn bộ hệ thống.
Phân quyền Truy cập (RBAC)	Áp dụng chính sách IAM Least Privilege chỉ cấp quyền Read/Write State cho CI/CD Runner.	Ngăn chặn truy cập trái phép hoặc thao tác sửa đổi thủ công vào file State.

Mô-đun và tái sử dụng mã hạ tầng

- Thành phần cốt lõi để nâng cấp kịch bản IaC từ các đoạn script đơn lẻ thành hệ thống quản trị hạ tầng quy mô Enterprise là **Mô-đun hóa (Modularization)**. Khái niệm này áp dụng nguyên lý **DRY (Don't Repeat Yourself)** trong kỹ nghệ phần mềm vào hạ tầng, đóng gói các khối tài nguyên liên quan thành một đơn vị tái sử dụng, độc lập và dễ kiểm soát.

Mô-đun và tái sử dụng mã hạ tầng



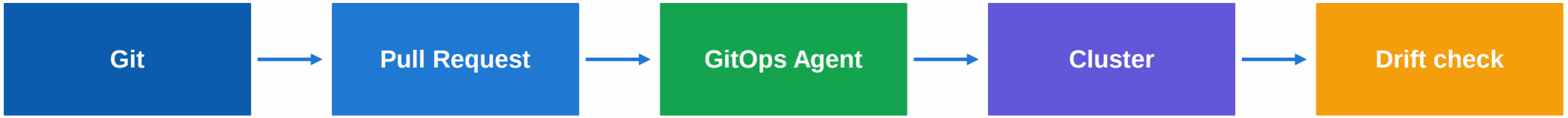
- Module đóng gói cấu hình hạ tầng thường dùng.
- Giúp chuẩn hóa mạng, máy chủ, database, logging và bảo mật.
- Giảm copy/paste và giảm sai lệch giữa dự án.
- Cần version module để tránh cập nhật gây lỗi dây chuyền.

Quản lý nhiều môi trường

- Dev, test, staging và production nên tách tài nguyên và quyền truy cập.
- Cấu hình dùng chung được module hóa; tham số thay đổi theo môi trường.
- Không dùng chung secret giữa môi trường học tập và production.
- Cần kiểm soát ai được áp dụng thay đổi lên production.



GitOps là gì?



- Git là nguồn sự thật mô tả trạng thái mong muốn của hệ thống.
- Thay đổi hạ tầng/ứng dụng đi qua pull request và review.
- Agent trong cụm theo dõi Git và đồng bộ trạng thái thực tế.
- Nếu cấu hình thực tế lệch khỏi Git, hệ thống phát hiện và khôi phục.

Phát hiện sai lệch cấu hình



- Drift xảy ra khi tài nguyên thực tế khác mã IaC/Git.
- Nguyên nhân thường do sửa thủ công trên console hoặc thay đổi ngoài pipeline.
- Drift làm mất khả năng tái lập và gây khó điều tra lỗi.
- Cần phát hiện, cảnh báo và đưa thay đổi quay lại quy trình quản lý mã.

Policy as Code



- Chính sách bảo mật và quản trị được mô tả bằng mã.
- Kiểm tra tự động trước khi tạo tài nguyên cloud.
- Ví dụ: không cho mở storage công khai, bắt buộc tag, bắt buộc mã hóa.
- Giúp chuẩn hóa tuân thủ và giảm phụ thuộc kiểm tra thủ công.

5.6. Khả năng quan sát và vận hành hệ thống

- Không thể vận hành hệ thống cloud hiệu quả nếu không đo lường và nhìn thấy trạng thái hệ thống.



Monitoring và Observability

- Monitoring trả lời: hệ thống có đang hoạt động bình thường không?
- Observability trả lời: vì sao hệ thống lại có trạng thái đó?
- Observability dựa trên log, metric, trace và ngữ cảnh triển khai.
- Ứng dụng phân tán cần khả năng truy vết qua nhiều dịch vụ.



Log



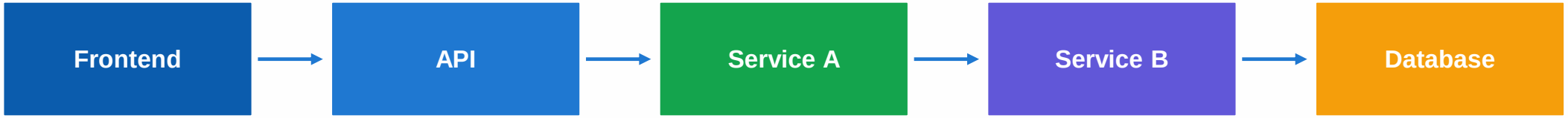
- Log ghi lại sự kiện xảy ra trong ứng dụng và hạ tầng.
- Log nên có thời gian, mức độ, request ID, người dùng hoặc dịch vụ liên quan.
- Không ghi mật khẩu, token hoặc dữ liệu nhạy cảm vào log.
- Log tập trung giúp truy vấn và điều tra sự cố nhanh hơn.

Metric



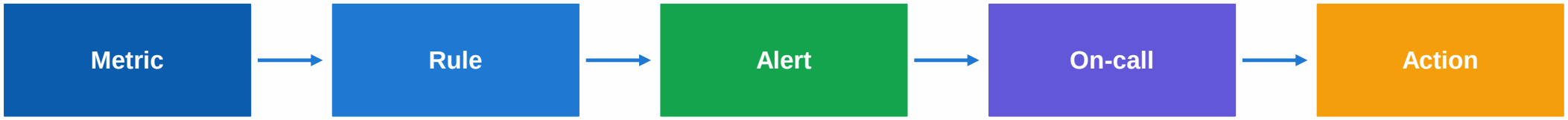
- Metric là số đo theo thời gian: CPU, memory, request, latency, error rate.
- Metric phù hợp dashboard, cảnh báo và phân tích xu hướng.
- Cần phân biệt metric hạ tầng và metric nghiệp vụ.
- Metric quá nhiều nhưng không có ngữ cảnh có thể gây nhiễu.

Distributed Trace



- Trace theo dõi một yêu cầu đi qua nhiều dịch vụ.
- Span biểu diễn một thao tác trong trace.
- Giúp tìm điểm nghẽn latency và lỗi trong kiến trúc microservices.
- Cần gắn correlation ID/request ID xuyên suốt chuỗi xử lý.

Dashboard và cảnh báo



- Dashboard trình bày trạng thái hệ thống theo thời gian thực.
- Cảnh báo cần dựa trên triệu chứng ảnh hưởng người dùng, không chỉ tài nguyên thấp/cao.
- Cần phân cấp cảnh báo: thông tin, cảnh báo, nghiêm trọng.
- Cảnh báo quá nhiều dẫn đến alert fatigue và bỏ sót sự cố thật.

SLI, SLO và Error Budget



- SLI là chỉ số đo chất lượng dịch vụ.
- SLO là mục tiêu chất lượng cần đạt.
- Error budget là phần sai số cho phép trong một khoảng thời gian.
- Khi error budget cạn, cần ưu tiên ổn định hơn phát hành tính năng mới.

Ứng phó sự cố vận hành



- Phát hiện: cảnh báo hoặc phản ánh người dùng.
- Phân loại: mức độ ảnh hưởng và phạm vi sự cố.
- Khắc phục: rollback, mở rộng, cô lập hoặc sửa cấu hình.
- Rút kinh nghiệm: postmortem không đổ lỗi và cải tiến hệ thống.

Theo dõi hiệu năng ứng dụng



- Đo thời gian phản hồi, throughput, error rate và saturation.
- Xác định bottleneck ở ứng dụng, database, network hoặc bên thứ ba.
- Dùng load test để kiểm tra trước các chiến dịch cao điểm.
- Tối ưu hiệu năng phải đi cùng kiểm soát chi phí.

5.7. Di chuyển và hiện đại hóa ứng dụng

- Di chuyển lên cloud cần kế hoạch kỹ thuật, dữ liệu, bảo mật, chi phí và tổ chức vận hành.



Đánh giá hiện trạng trước khi di chuyển

- Lập danh mục ứng dụng, máy chủ, database, dữ liệu và phụ thuộc.
- Đánh giá yêu cầu bảo mật, tuân thủ, độ trễ và thời gian dừng cho phép.
- Xác định chi phí hiện tại và chi phí cloud dự kiến.
- Phân loại ứng dụng theo mức độ ưu tiên và rủi ro.



Bảy chiến lược di chuyển

- **Di chuyển và hiện đại hóa ứng dụng (Cloud Migration & Modernization)** là hành trình chuyển dịch hạ tầng, phần mềm và dữ liệu từ trung tâm dữ liệu truyền thống (On-Premises) sang môi trường Điện toán đám mây. Khung chiến lược được áp dụng chuẩn hóa nhất trên toàn cầu hiện nay là **Khung 7 Rs (The 7 Rs of Cloud Migration)** do Gartner và AWS phát triển.

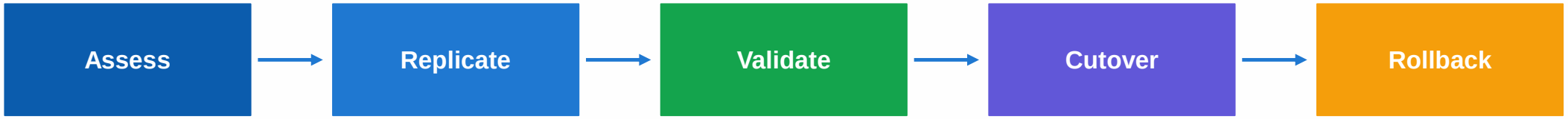
Bảng So sánh Tổng quan 7 Chiến lược Di chuyển (7 Rs)

Chiến lược (7 Rs)	Mức độ thay đổi Code	Chi phí & Thời gian ban đầu	Mức độ tối ưu Cloud	Trường hợp sử dụng phù hợp
1. Rehost (Lift & Shift)	**Không**	Rất thấp / Rất nhanh	Thấp (Chỉ đổi hạ tầng)	Cần đóng cửa Data Center gấp; App legacy ổn định.
2. Relocate	**Không**	Rất thấp / Tức thì	Thấp (Giữ nguyên hypervisor)	Chuyển cụm VMware/K8s On-Premises lên Cloud VMware/EKS.
3. Replatform (Lift & Reshape)	**Tối thiểu**	Trung bình	Trung bình	Đổi DB self-hosted sang Cloud Managed DB (RDS/Cloud SQL).
4. Refactor / Re-architect	**Rất cao** (Viết lại)	Cao / Tốn thời gian	**Tối đa** (Cloud-Native)	Hệ thống cốt lõi (Core Business) cần scale cực cao.
5. Repurchase (Drop & Replace)	**Loại bỏ code cũ**	Trung bình	Cao (SaaS-native)	Thay thế ứng dụng tự viết (HR, ERP, Mail) bằng SaaS.
6. Retain (Keep / Do Nothing)	**Không**	Không	Không	Ứng dụng mới nâng cấp, chưa đến lúc hoặc phụ thuộc phần cứng.
7. Retire (Decommission)	**Xóa bỏ**	Không	Không	Ứng dụng dư thừa, không còn người sử dụng.

So sánh nhanh các chiến lược migration

Chiến lược (7 Rs)	Nỗ lực / Sửa Code	Tốc độ Triển khai	Lợi ích Cloud	Chi phí Ban đầu	Trường hợp Tối ưu
Rehost *(Lift & Shift)*	Không sửa code	**Cực nhanh**	Thấp	Rất thấp	Cần đóng Data Center gấp, App legacy ổn định.
Relocate	Không sửa code	**Tức thì**	Thấp	Rất thấp	Chuyển nguyên cụm VMware/K8s On-Premises lên Cloud.
Replatform *(Lift & Reshape)*	Sửa nhẹ / Cấu hình	Nhanh	Trung bình	Thấp – Vừa	Chuyển DB tự cài sang Cloud Managed DB (RDS, Cloud SQL).
Refactor *(Re-architect)*	**Viết lại toàn bộ**	Chậm	**Tối đa**	Rất cao	Core App cần scale linh hoạt, chuẩn Microservices/Serverless.
Repurchase *(Drop & Replace)*	Bỏ hẳn code cũ	Nhanh	Cao	Thuê bao SaaS	Chuyển sang dùng SaaS thương mại (Salesforce, M365).
Retain *(Keep)*	Không can thiệp	-	Không	0	App mới nâng cấp, vướng quy định bảo mật/dữ liệu.
Retire *(Decommission)*	Xóa / Tắt hoàn toàn	Nhanh	Tiết kiệm 100%	0	App dư thừa, không còn người sử dụng.

Di chuyển cơ sở dữ liệu



- Phân tích dung lượng, downtime cho phép, yêu cầu nhất quán và bảo mật.
- Chọn phương án backup/restore, replication hoặc migration service.
- Kiểm thử hiệu năng truy vấn và tương thích ứng dụng.
- Lập kế hoạch cutover và rollback cho dữ liệu.

Kiểm thử và cutover



- Kiểm thử chức năng, hiệu năng, bảo mật và khôi phục sự cố.
- Chạy thử song song nếu hệ thống quan trọng.
- Cutover cần thời điểm phù hợp và kế hoạch truyền thông.
- Theo dõi sát sau cutover để phát hiện lỗi sớm.

Kế hoạch rollback



- Xác định điều kiện kích hoạt rollback trước khi triển khai.
- Bảo toàn dữ liệu phát sinh trong thời gian chuyển đổi.
- Chuẩn bị hạ tầng cũ hoặc phương án phục hồi tạm thời.
- Thực hành rollback trong môi trường kiểm thử nếu có thể.

Hybrid cloud và Multi-cloud



- Hybrid cloud kết hợp on-premise/private cloud với public cloud.
- Multi-cloud dùng nhiều nhà cung cấp cloud khác nhau.
- Có thể tăng linh hoạt nhưng cũng tăng độ phức tạp quản trị.
- Cần chuẩn hóa identity, network, logging, policy và chi phí.

5.8. Quản trị và tối ưu chi phí - FinOps

- Chi phí cloud cần được đo lường, phân bổ và tối ưu liên tục theo giá trị sử dụng.



Vì sao cần FinOps?

- Cloud chuyển chi phí từ đầu tư cố định sang chi phí vận hành biến động.
- Tài nguyên có thể tạo nhanh nên chi phí cũng tăng nhanh.
- Cần phối hợp giữa kỹ thuật, tài chính và quản lý sản phẩm.
- Mục tiêu là tối đa hóa giá trị trên mỗi đồng chi tiêu cloud.



Gắn nhãn và phân bổ chi phí



- Tag/label giúp biết tài nguyên thuộc dự án, nhóm, môi trường nào.
- Phân bổ chi phí giúp nhóm chịu trách nhiệm với tài nguyên mình dùng.
- Thiếu tag làm khó báo cáo, tối ưu và kiểm soát ngân sách.
- Nên quy định bộ tag bắt buộc: owner, project, environment, cost-center.

Ngân sách và cảnh báo chi phí



- Thiết lập ngân sách theo dự án, môi trường hoặc dịch vụ.
- Cảnh báo khi chi phí đạt ngưỡng 50%, 80%, 100%.
- Theo dõi bất thường chi phí để phát hiện cấu hình sai hoặc tấn công.
- Không đợi cuối tháng mới kiểm tra hóa đơn.

Rightsizing tài nguyên



- So sánh tài nguyên cấp phát với mức sử dụng thực tế.
- Giảm kích thước máy ảo, database hoặc container nếu dư thừa lâu dài.
- Tăng tài nguyên khi thiếu để tránh ảnh hưởng trải nghiệm người dùng.
- Quyết định rightsizing cần dựa trên dữ liệu quan sát đủ dài.

Mô hình giá cloud



- On-demand: linh hoạt, trả theo mức dùng.
- Reserved/Committed use: rẻ hơn nếu cam kết dùng lâu dài.
- Spot/Preemptible: rẻ nhưng có thể bị thu hồi tài nguyên.
- Serverless: tính phí theo request, thời gian chạy hoặc tài nguyên thực dùng.

Unit economics và bền vững



- Đo chi phí trên mỗi người dùng, giao dịch, lớp học hoặc đơn hàng.
- • Tối ưu chi phí dựa trên giá trị kinh doanh, không chỉ tổng hóa đơn.
- • Tắt tài nguyên không dùng và chọn vùng/loại tài nguyên phù hợp.
- • Tối ưu tài nguyên cũng góp phần giảm tiêu thụ năng lượng.

5.9. Trí tuệ nhân tạo trên điện toán đám mây

- Cloud cung cấp hạ tầng, dữ liệu và dịch vụ được quản lý để phát triển AI/ML và AI tạo sinh.



Dịch vụ AI/ML được quản lý

- Cloud cung cấp dịch vụ chuẩn bị dữ liệu, huấn luyện, triển khai và giám sát mô hình.
- Dịch vụ được quản lý giúp giảm công việc vận hành hạ tầng AI.
- Có thể dùng API AI sẵn có cho nhận dạng ảnh, giọng nói, ngôn ngữ hoặc chatbot.
- Cần đánh giá quyền riêng tư, bảo mật dữ liệu và chi phí inference.

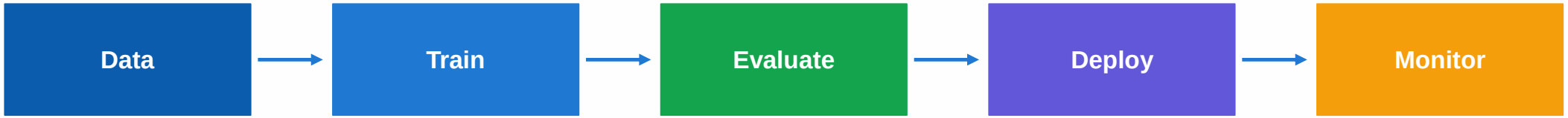


GPU và bộ tăng tốc AI



- Huấn luyện và suy luận mô hình lớn thường cần GPU hoặc máy gia tốc (accelerator).
- Tài nguyên AI có chi phí cao và cần lập kế hoạch sử dụng cẩn thận.
- Có thể dùng notebook, VM GPU, container GPU hoặc dịch vụ đào tạo được quản lý (managed training).
- Cần theo dõi utilization để tránh lãng phí tài nguyên đắt tiền.

Quy trình MLOps cơ bản



- Quản lý dữ liệu, mã huấn luyện, tham số và phiên bản mô hình.
- Tự động kiểm thử, huấn luyện, đánh giá và triển khai mô hình.
- Giám sát drift dữ liệu, chất lượng dự đoán và hiệu năng dịch vụ.
- Rollback hoặc thay thế mô hình khi chất lượng giảm.

AI tạo sinh, chatbot và RAG



- AI tạo sinh có thể hỗ trợ hỏi đáp, tóm tắt, sinh nội dung và trợ lý ảo.
- RAG kết hợp mô hình ngôn ngữ với kho tri thức riêng của tổ chức.
- Cơ sở dữ liệu vector hỗ trợ tìm kiếm ngữ nghĩa.
- Cần kiểm soát dữ liệu nhạy cảm, ảo giác (hallucination), logging và chi phí token.

Bảo mật và chi phí cho AI trên cloud



- Không gửi dữ liệu nhạy cảm vào dịch vụ AI nếu chưa có chính sách rõ ràng.
- Kiểm soát quyền truy cập dữ liệu huấn luyện và dữ liệu truy vấn.
- Ghi log phải tránh lưu nội dung riêng tư không cần thiết.
- Theo dõi chi phí token, GPU, storage và pipeline xử lý dữ liệu.

Bài tập

- Sinh viên thiết kế, triển khai thử nghiệm và thuyết trình một giải pháp cloud hoàn chỉnh.



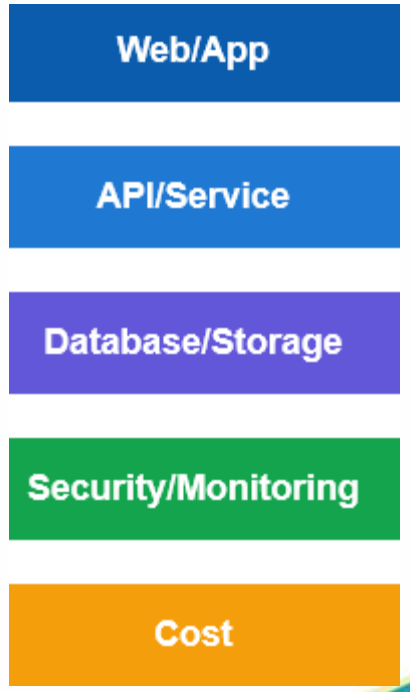
Yêu cầu bài tập



- Thiết kế một ứng dụng hoặc hệ thống dịch vụ chạy trên cloud.
- Có sơ đồ kiến trúc, phân tích mô hình dịch vụ và mô hình triển khai.
- Có thành phần compute, network, storage/database, bảo mật và giám sát.
- Có minh chứng triển khai hoặc mô phỏng cấu hình.
- Có phân tích chi phí, rủi ro và hướng mở rộng.

Chủ đề gợi ý

- Website thương mại điện tử ba lớp trên cloud.
- Hệ thống quản lý học tập hoặc quản lý tài liệu.
- Ứng dụng serverless xử lý ảnh hoặc gửi thông báo.
- Microservices đơn giản triển khai bằng container/Kubernetes.
- Chatbot dùng dịch vụ AI cloud và cơ sở dữ liệu vector.
- Hệ thống sao lưu, giám sát và phục hồi sau sự cố.



Sản phẩm bàn giao



- Báo cáo thiết kế kiến trúc và quyết định kỹ thuật.
- Sơ đồ kiến trúc rõ các lớp: người dùng, mạng, ứng dụng, dữ liệu, bảo mật.
- Cấu hình triển khai: script, manifest, IaC hoặc ảnh minh chứng console.
- Slide thuyết trình và phần demo ngắn.
- Phân tích bảo mật, sẵn sàng, mở rộng và chi phí.

Tài liệu và hướng tự học



- Ôn lại Chương 1–4 để liên kết dịch vụ, hạ tầng, dữ liệu và bảo mật.
- Tự thực hành container hóa một ứng dụng web đơn giản.
- Tìm hiểu lệnh kubectl cơ bản: get, describe, logs, apply, rollout.
- Thử mô phỏng pipeline CI/CD và vẽ kiến trúc bài tập lớn.
- Chuẩn bị câu hỏi cho buổi báo cáo/thảo luận nhóm.

Thank you!

